

Relazione di Laboratorio: PANICO ALLA PIZZA!

Mini-Progetto di Videogioco in Python con Pygame

1. Dati del Progetto

Studenti	Sofia Grygoryshena, Giulia Dicanio
Classe	3 BTLC
Titolo del gioco	Panico Alla Pizza
Linguaggio / Libreria	Python 3 + Pygame
Data di consegna	03/06/2026

2. Idea del Gioco

Panico Alla Pizza è un gioco di abilità ("catcher game") a scorrimento verticale. Due oggetti cadono dal cielo verso il basso: pizze da raccogliere per guadagnare punti e bombe da evitare per non perdere vite. Di tanto in tanto compaiono anche stelle dorate che offrono un bonus di punti più alto. Il giocatore controlla il personaggio con i tasti freccia sinistra e destra e deve sopravvivere il più a lungo possibile.

All'inizio il giocatore sceglie uno dei due personaggi disegnati a mano nel codice: Sofia (capelli biondi, vestito blu) oppure Giulia (capelli ricci scuri, vestito verde). Questa scelta è puramente estetica e non influisce sulla difficoltà di gioco.

L'originalità rispetto a un semplice esempio di catcher trovato online sta in diversi elementi: i personaggi sono illustrazioni custom disegnate con le primitive di Pygame (cerchi, poligoni, linee), il gioco include musica di sottofondo generata proceduralmente in codice (sintesi 8-bit), un sistema di particelle per feedback visivo, un'animazione del lampeggio degli occhi e del movimento delle braccia, oltre a un sistema di livelli progressivo che aumenta la velocità e la frequenza di comparsa degli oggetti. Il pubblico immaginato sono coetanei o ragazzi/ragazze che conoscono le due protagoniste, data la personalizzazione dei personaggi con i loro nomi.

3. Analisi e Progettazione

3.1 Regele del Gioco

- Il giocatore parte con 3 vite.
- Raccogliere una pizza vale +10 punti per il livello corrente.
- Raccogliere una stella vale +50 punti per il livello corrente.
- Toccare una bomba fa perdere 1 vita. Il personaggio lampeggia brevemente (ha un'invincibilità di 60 frame).
- Quando le vite arrivano a 0, il gioco passa allo stato GAME OVER.
- Il livello aumenta automaticamente al raggiungimento di soglie di punteggio: 100, 250, 400, 600 e 1000 punti.
- All'aumentare del livello, la velocità di caduta degli oggetti e la frequenza di spawn aumentano.
- Il record della sessione viene salvato in memoria e mostrato nel menu e nella schermata di game over.
- Si può ricominciare premendo R (si torna al menu di selezione personaggio).

3.2 Struttura del Programma

Il programma è contenuto in un unico file .py e si divide nelle seguenti aree logiche:

- **Inizializzazione (init):** si avvia Pygame, si crea la finestra 800x600, si caricano i font di sistema (Comic Sans MS con fallback) e si tenta l'inizializzazione del mixer audio.
- **Generazione audio procedurale:** i suoni (cattura pizza, stella, esplosione bomba e musica di sottofondo) vengono sintetizzati direttamente nel codice usando onde sinusoidali e rumore, senza file audio esterni.

- **Funzioni grafiche helper:** draw_text(), draw_rounded_rect(), lerp_color() per il rendering del testo con ombra, rettangoli arrotondati semitrasparenti e interpolazione dei colori.
- **Funzioni di disegno personaggi/oggetti:** draw_sofia(), draw_giulia(), draw_pizza(), draw_bomb(), draw_star(), draw_background(). Tutte usano le primitive Pygame senza immagini esterne.
- **Classi principali:** Particle, FallingObj, Player.
- **Ciclo di gioco principale (main()):** gestisce tre stati STATE_SELECT, STATE_PLAY, STATE_DEAD con la relativa logica di input, aggiornamento e rendering per ciascuno.

Schema a blocchi semplificato del ciclo di gioco:

1. Avvio programma
2. Mostra schermata selezione personaggio (STATE_SELECT)
 - Il giocatore preme 1 o 2 oppure clicca sul personaggio.
3. Inizializza Player, svuota liste oggetti/particelle
4. Ciclo di gioco (STATE_PLAY):
 - Calcola nuovo livello in base al punteggio.
 - Incrementa timer spawn e crea nuovo FallingObj se scaduto.
 - Player.move(keys) -> aggiorna posizione.
 - Per ogni FallingObj: aggiorna posizione e controlla collisioni.
 - **PIZZA:** +punti, particelle arancioni, suono pizza.
 - **BOMBA:** -1 vita, particelle rosse, flash schermo, suono bomba.
 - **STELLA:** +bonus, particelle dorate, suono stella.
 - Aggiorna e disegna particelle.
 - Disegna HUD (punteggio, vite, livello).
 - Se vite < 0 -> imposta STATE_DEAD.
5. Schermata GAME OVER (STATE_DEAD)
 - Pressione del tasto R per tornare a STATE_SELECT.

3.3 Strutture Dati Importanti

Il programma utilizza tre classi principali e alcune liste globali di supporto:

- **Classe Player:** rappresenta il personaggio controllato dal giocatore. Contiene attributi per posizione (x, y), vite, punteggio, stato di movimento, timer per il lampeggio degli occhi (blink_timer), timer di invincibilità dopo una bomba (hurt_timer) e testo del feedback cattura (caught_text). I metodi principali sono move(keys), update() e draw(surf).

- **Classe FallingObj:** rappresenta un singolo oggetto che cade (pizza, bomba o stella). Ogni istanza tiene traccia del tipo (kind), posizione, velocità di caduta, angolo di rotazione e un timer interno (t) per animazioni come il tremore della miccia. Il metodo update() fa avanzare l'oggetto verso il basso con un leggero oscillare laterale (funzione seno). Il metodo rect() restituisce il rettangolo di collisione.
 - **Classe Particle:** rappresenta un singolo elemento del sistema di particelle. Ogni particella ha una posizione iniziale, velocità casuale, colore, vita residua (life) e un testo opzionale (es. "+10", "AH!!"). Il metodo update() applica la gravità e riduce la vita; draw() la disegna con trasparenza decrescente (fade-out).
 - **Lista objects:** contiene tutti i FallingObj attualmente sullo schermo. Gli oggetti vengono aggiunti periodicamente da spawn_object() e rimossi quando escono dallo schermo o vengono raccolti/colpiti dal giocatore.
 - **Lista particles:** contiene tutte le particelle attive. Viene aggiornata ogni frame e gli elementi con life ≤ 0 vengono rimossi.
-

4. Sviluppo e Uso dell'AI

4.1 Come abbiamo lavorato

Abbiamo lavorato in coppia dividendo le responsabilità principali: una di noi si è concentrata sulla logica di gioco (collisioni, spawn, livelli) mentre l'altra ha curato la parte grafica (disegno dei personaggi, animazioni, schermata di selezione). La parte che ha richiesto più tempo è stata il disegno dei personaggi con le primitive Pygame, perché abbiamo dovuto aggiustare molte coordinate manualmente per ottenere un risultato soddisfacente. Anche la generazione audio procedurale è risultata più complessa del previsto, poiché richiede una comprensione delle onde sonore digitali.

4.2 Come abbiamo usato l'AI

Abbiamo utilizzato Claude (Anthropic) come principale assistente AI durante lo sviluppo. In particolare, l'AI ci ha aiutato nei seguenti ambiti:

- Generazione della struttura iniziale del ciclo di gioco (game loop con stati).
- Scrittura delle funzioni di disegno dei personaggi (coordinate dei poligoni e cerchi).
- Implementazione della sintesi audio procedurale (bytearray con onde sinusoidali).

- Correzione di errori come `AttributeError` su `pygame.SRCALPHA` e gestione del mixer.
- Refactoring del codice per introdurre le classi `Player`, `FallingObj` e `Particle`.

Alcuni prompt principali utilizzati (esempi indicativi):

- *"Scrivi in Python con Pygame un gioco in cui oggetti cadono dall'alto e il giocatore li raccoglie muovendosi a sinistra e destra."*
- *"Come posso disegnare un personaggio femminile con i capelli biondi usando solo `pygame.draw`?"*
- *"Ho questo errore: `pygame.error: No available audio device`. Come gestisco il caso in cui l'audio non sia disponibile?"*
- *"Aggiungi un sistema di particelle che mostri +10 quando il giocatore prende una pizza."*

Abbiamo sempre riletto e adattato il codice generato: abbiamo modificato i nomi delle variabili, aggiunto commenti, cambiato i colori e le proporzioni dei personaggi per renderli più simili a noi, e integrato le parti generate in modo coerente con il resto del codice.

5. Test e Risultati

5.1 Casi di Test

#	Cosa abbiamo fatto	Risultato atteso	Risultato ottenuto
1	Il personaggio tocca una bomba	Perde 1 vita, lampeggia	Funziona correttamente
2	Il personaggio tocca una pizza	+10 punti (liv.1), particelle arancioni	Funziona correttamente

3	Raggiunge 100 punti	Passa al livello successivo, comparsa particelle dorate	Funziona correttamente
4	Vite arrivano a 0	Schermata GAME OVER con punteggio	Funziona correttamente
5	Pressione di R nel GAME OVER	Ritorno al menu di selezione personaggio	Funziona correttamente
6	Raccolta di una stella	+50 punti (liv.1), particelle dorate	Funziona correttamente
7	Avvio su PC senza altoparlanti	Il gioco parte senza crash audio	Funziona (try/except gestito)
8	Punteggio >= 1000	Livello 6, caduta molto più veloce	Funziona correttamente

5.2 Bug e Problemi Noti

- Il record del punteggio viene mantenuto solo durante la sessione corrente: chiudendo e riaprendo il gioco il record si azzerà, perché non è implementato il salvataggio su file.
- Su alcune macchine senza driver audio, il mixer audio genera un avviso nella console (non un crash). Il gioco funziona comunque, ma senza suono. Questo comportamento è gestito intenzionalmente con un blocco try/except.
- Gli oggetti che cadono possono a volte sovrapporsi tra loro visivamente se spawnati nella stessa posizione x in frame vicini. Non compromette la giocabilità ma è esteticamente migliorabile.

6. Conclusioni e Riflessione Personale

Questo progetto ci ha permesso di approfondire molti aspetti di Python che non avevamo ancora usato in modo pratico: le classi con metodi e attributi, i cicli di gioco con stati, la gestione degli eventi Pygame e la matematica per le animazioni (funzione seno per il movimento ondulatorio, interpolazione di colori). Abbiamo anche imparato che la generazione di suoni digitali si basa su formule matematiche applicate alle onde sinusoidali.

Disegnare i personaggi con le sole primitive grafiche di Pygame è stato molto laborioso: ogni minima modifica richiedeva di ricalcolare coordinate e misure manualmente. Abbiamo impiegato molto tempo per ottenere proporzioni accettabili. Per superare questa difficoltà, abbiamo usato carta e penna per schizzare i personaggi e le loro proporzioni prima di tradurli in coordinate nel codice. Per gli errori di programmazione, invece, l'AI è stata fondamentale per comprendere al meglio i messaggi di errore e correggere rapidamente i bug.

Se rifacessimo il progetto da capo, probabilmente useremmo immagini PNG per i personaggi invece del disegno procedurale, ottimizzando sensibilmente le tempistiche di sviluppo. Implementeremmo inoltre il salvataggio persistente del record su file fin dall'inizio. Riteniamo di aver usato l'AI in modo consapevole: non abbiamo incollato codice in modo passivo, ma abbiamo sempre analizzato e riadattato le risposte. In alcuni momenti ci siamo appoggiate maggiormente al supporto dell'AI per le parti più complesse (come la sintesi audio), che avremmo fatto fatica a sviluppare interamente da sole.

7. Allegati

- **File sorgente:** `panico_alla_pasta.py` (consegnato separatamente)
- **Screenshot della schermata di selezione personaggio** (consegnato separatamente)
- **Screenshot della schermata di gioco in esecuzione** (consegnato separatamente)
- **Screenshot della schermata di GAME OVER** (consegnato separatamente)

Nota: Il codice sorgente è interamente commentato nelle sue sezioni principali (inizializzazione, generazione audio, disegno dei personaggi, classi e game loop). Si consiglia di consultarlo unitamente a questa relazione per una comprensione completa del progetto.